CAMBRIDGE 2210
CHAPTER 01

CAMBRIDGE O LEVEL · COMPUTER SCIENCE (2210)

Chapter 1
Data Representation

Professional Exam Revision Notes

CREATED
July 2026

TARGET GRADE
A* Exam Preparation

COVERAGE
All examinable content — Chapter 1

SYLLABUS
Cambridge O Level 2210

Learning Objectives

🔑 **WHAT THIS CHAPTER COVERS**

- How computers use binary to represent all data
- Conversion between denary, binary and hexadecimal number systems
- Real-world applications of hexadecimal (error codes, MAC addresses, IPv6, HTML colours)
- Binary arithmetic including addition and logical shifts
- Two's complement representation of negative numbers
- How text, sound and images are encoded digitally
- Data storage measurement (IEC binary system)
- Data compression techniques (lossy and lossless)

SECTION 1.1

Number Systems

1.1.1 Why Binary?

📘 **KEY CONCEPT**

Computers use binary because they contain **millions of switches** that are either ON or OFF. ON = 1, OFF = 0. Binary is the **only** number system that directly corresponds to the physical nature of computer hardware.

⚠️ **IMPORTANT**

All data processed by a computer must be converted to binary format.

1.1.2 The Three Number Systems

Binary (Base 2)

Uses only digits **0** and **1**

Each position represents a power of 2: 2^0, 2^1, 2^2, 2^3, etc.

8-bit binary headings: 128, 64, 32, 16, 8, 4, 2, 1

Denary (Base 10)

Uses digits **0–9**

The number system we use daily

Each position represents a power of 10

Hexadecimal (Base 16)

Uses digits 0–9 and letters **A–F**

A=10, B=11, C=12, D=13, E=14, F=15

Each position is a power of 16: 16^0, 16^1, 16^2, etc.

⚠️ **CRITICAL RELATIONSHIP**

$16 = 2^4$, so **ONE hex digit = FOUR binary digits**.

Binary to Denary Conversion Table

Binary	Hex	Denary	Binary	Hex	Denary
0000	0	**0**	1000	8	**8**
0001	1	**1**	1001	9	**9**
0010	2	**2**	1010	A	**10**
0011	3	**3**	1011	B	**11**
0100	4	**4**	1100	C	**12**
0101	5	**5**	1101	D	**13**
0110	6	**6**	1110	E	**14**
0111	7	**7**	1111	F	**15**

Converting Between Number Systems

Binary to Denary

> 📘 **METHOD**
> Add the column values where there is a 1.

128	64	32	16	8	4	2	1
1	**1**	**1**	0	**1**	**1**	**1**	0

= 128 + 64 + 32 + 8 + 4 + 2 = **238 denary**

DENARY → BINARY · METHOD 1 (SUBTRACTION)

Repeatedly subtract the largest power of 2:

```
142 = 128 + 14
    = 128 + 8 + 4 + 2
    = 10001110 (8-bit binary)
```

DENARY → BINARY · METHOD 2 (DIVISION)

```
142 ÷ 2 = 71 r0
 71 ÷ 2 = 35 r1
 35 ÷ 2 = 17 r1
 17 ÷ 2 =  8 r1
  8 ÷ 2 =  4 r0
  4 ÷ 2 =  2 r0
  2 ÷ 2 =  1 r0
  1 ÷ 2 =  0 r1
```

Read bottom-to-top: **10001110**

BINARY → HEXADECIMAL

Split binary into groups of 4 bits (from right), convert each group using the reference table.

```
1011 1110 0001  =  B E 1 (hex)
```

HEXADECIMAL → BINARY

Convert each hex digit to 4-bit binary.

```
2  1  F  D
0010 0001 1111 1101
```

HEXADECIMAL → DENARY

Multiply each digit by its column value (4096, 256, 16, 1).

```
4 5 A (hex)
= (4×256)+(5×16)+(10×1)
= 1024 + 80 + 10 = 1114 denary
```

DENARY → HEXADECIMAL

Repeatedly divide by 16, read remainders bottom-to-top.

```
2004 ÷ 16 = 125 r4
 125 ÷ 16 =   7 r13 (D)
   7 ÷ 16 =   0 r7
= 7D4 (hex)
```


1.1.3 Why Use Hexadecimal?

Advantages	Disadvantages
More compact than binary (1 hex digit = 4 binary digits)	Not as intuitive as decimal for everyday use
Easier for humans to remember and write	Less familiar to non-programmers
Less error-prone than long binary strings	Requires knowledge of A–F digit values
Directly related to binary by a power of 2	—

📘 1. ERROR CODES

Generated automatically by operating systems. Refer to memory addresses where errors occur. Example: **`0x0000007B`** (Windows Blue Screen of Death). Windows displays errors as hexadecimal for programmers to diagnose.

📘 2. MAC ADDRESSES (MEDIA ACCESS CONTROL)

Uniquely identifies network devices. Format: `NN-NN-NN-DD-DD-DD` (48 bits total)

- First 3 pairs (NN-NN-NN): Manufacturer ID
- Last 3 pairs (DD-DD-DD): Device serial number

Example: `00-1C-B3-4F-25-FE` (Apple Corp device with serial 4F25FE)

Other manufacturers: Dell (00-14-22), Cisco (00-40-96), Intel (00-a0-c9)

📘 3. IPV6 ADDRESSES

New internet protocol using 128-bit addresses. Split into 8 groups of 16 bits (4 hex digits each). Written with colons:

`a8fb:7a88:fff0:0fff:3d21:2085:66fb:f0fa`

IPv4 (older) uses decimal: `109.108.158.1`

📘 4. HTML COLOUR CODES

Represents colours in web design. Format: #RRGGBB (6 hex digits)

- `#FF0000` = Red (Red=FF, Green=00, Blue=00)
- `#00FF00` = Green
- `#0000FF` = Blue
- `#FFFFFF` = White
- Possible colours: 256 × 256 × 256 = **16,777,216 colours**

1.1.4 Binary Addition

Rules for Adding Two Binary Digits

Operation	Result	Carry
0 + 0	0	0
0 + 1	1	0
1 + 0	1	0
1 + 1	0	1

```
  00100111  (39 denary)
+ 01001010  (74 denary)
------------
  01110001  (113 denary)
```

Working right to left:

- Column 1: 1 + 0 = 1, no carry
- Column 2: 1 + 1 = 0, carry 1
- Column 3: 1 + 0 + 1(carry) = 0, carry 1
- Column 4: 0 + 1 + 1(carry) = 0, carry 1
- Column 5: 0 + 0 + 1(carry) = 1, no carry

⚠️ OVERFLOW ERROR

Definition: When the result of binary addition requires more bits than available to store it. In 8-bit systems, maximum value = $2^8 - 1 = 255$ denary. If addition produces a 9th bit, an overflow error occurs and the result is incorrect.

```
  01101110  (110 denary)
+ 11011110  (222 denary)
------------
1)01001100  (result claims 76, but actually 332)
↑ This 9th bit is lost — OVERFLOW ERROR
```

1.1.5 Logical Binary Shifts

📘 **DEFINITION**

Moving all bits in a binary number left or right, filling empty positions with 0s.

Effects

Left Shift: multiplies the number by 2 for each position shifted (×2, ×4, ×8, etc.)

Right Shift: divides the number by 2 for each position shifted (÷2, ÷4, ÷8, etc.)

EXAMPLE — LEFT SHIFT

```
Original:      00010101  (21)
Shift 1 left:  00101010  (42)  = 21 × 2
Shift 2 left:  01010100  (84)  = 21 × 4
Shift 3 left:  10101000  (168) = 21 × 8
```

EXAMPLE — RIGHT SHIFT

```
Original:       11001000  (200)
Shift 1 right:  01100100  (100) = 200 ÷ 2
Shift 2 right:  00110010  (50)  = 200 ÷ 4
Shift 3 right:  00011001  (25)  = 200 ÷ 8
```


1.1.6 Two's Complement (Representing Negative Numbers)

📘 **PURPOSE**

Allows binary numbers to represent both positive **and** negative integers.

8-bit Two's Complement Headings

−128	64	32	16	8	4	2	1
.	.	.	.	.	.	.	.

Note: the leftmost bit is negative (−128 instead of +128).

🔑 **KEY RULE**

- Leftmost bit = 1 → **negative** number
- Leftmost bit = 0 → **positive** number
- Range: **−128 to +127** (for 8-bit)
- Range: **−32,768 to +32,767** (for 16-bit)

Converting Negative Denary to Two's Complement

METHOD 1 — DIRECT

Negative number needs 1 in the −128 column; build the remaining value from the other headings.

METHOD 2 — INVERSION

```
1. 67 → 01000011  (positive binary)
2. Invert bits →  10111100
3. Add 1       →  10111101
```

✅ **RESULT**

Both methods give the same result for **−67**: `10111101`

CONVERTING NEGATIVE BINARY TO DENARY

Convert `10010011` (two's complement) to denary. Leftmost bit is 1 (negative).

```
−128 + 16 + 2 + 1 = −109 denary
```

Text, Sound and images

1.2.1 Character Sets: ASCII and Unicode

ASCII Code (American Standard Code for Information Interchange)

📘 STANDARD ASCII

- 7-bit codes (0–127 denary, 00–7F hex)
- Established 1963, updated 1986
- Represents letters, numbers, keyboard characters, control codes
- Control codes: 0–31 (for printer/computer control, not printable)

📘 EXTENDED ASCII

- 8-bit codes (0–255 denary, 0–FF hex)
- Adds 128 additional characters for non-English alphabets and graphics
- Different operating systems use different extended ASCII tables

⚠️ DISADVANTAGE

Only covers Western languages. Cannot represent Chinese, Japanese, Arabic, Cyrillic, etc.

Unicode

📘 PURPOSE

Universal character encoding for all world languages and writing systems.

🔑 KEY FEATURES

- Supports all languages (English, Russian, Chinese, Arabic, etc.)
- Uses 1–4 bytes per character (vs. 1 byte for ASCII)
- First 128 characters identical to ASCII (backward compatible)
- Can represent tens of thousands of characters
- Established 1991, Version 1.0 published

✅ ADVANTAGES

Supports all global languages; a single standard eliminates compatibility issues.

1.2.2 Representation of Sound

📘 PROBLEM & SOLUTION

Problem: Sound is analogue (continuous wave). Computers can only process digital (discrete) data.

Solution: Convert analogue to digital using sampling with an Analogue-to-Digital Converter (ADC).

Key Sampling Parameters

Parameter	Definition	Effect on File Size	Effect on Quality
Sampling Rate (Frequency)	Number of samples per second (Hz)	Higher rate = Larger file	Higher rate = Better sound
Sampling Resolution (Bit Depth)	Bits per sample (amplitude levels)	Higher resolution = Larger file	Higher resolution = Better quality

💡 **CD QUALITY STANDARD**

Sampling rate: **44.1 kHz** (44,100 samples per second) + Sampling resolution: **16 bits per sample** = high-quality audio that meets professional standards.

1.2.3 Representation of Bitmap Images

📘 **DEFINITION**

Bitmap images are made up of **pixels** (picture elements) arranged in a 2D matrix (rows × columns). Each pixel stores colour information as a binary number.

Colour Depth

📘 **DEFINITION**

Number of bits used to represent each pixel's colour.

Bits	Colours	Bits	Colours
1 bit	2 colours	16 bits	65,536 colours
2 bits	4 colours	24 bits	16,777,216 colours
4 bits	16 colours	32 bits	4,294,967,296 colours
8 bits	256 colours	—	—

🔑 **RGB MODEL (24-BIT COLOUR)**

8 bits for Red (0–255) + 8 bits for Green (0–255) + 8 bits for Blue (0–255) = 256 × 256 × 256 = **16,777,216 colours possible**.

Image Resolution

📘 **DEFINITION**

Total number of pixels in an image (width × height). Example: 4096 × 3072 pixels = 12,582,912 total pixels.

- **High resolution:** more detail, sharper image, LARGER file
- **Low resolution:** less detail, pixelated/fuzzy appearance, SMALLER file

CALCULATING BITMAP IMAGE FILE SIZE — FORMULA

```
File size (bits)  = width × height × colour depth
File size (bytes) = (width × height × colour depth) ÷ 8
File size (MiB)   = (width × height × colour depth) ÷ 8 ÷ 1,048,576
```

WORKED EXAMPLE – 2048 × 1024 PIXELS, 8 BITS PER PIXEL

```
Total pixels = 2,048 × 1,024        = 2,097,152
Total bits   = 2,097,152 × 8        = 16,777,216 bits
In bytes     = 16,777,216 ÷ 8       = 2,097,152 bytes
In MiB       = 2,097,152 ÷ 1,048,576 = 2 MiB
```

Data Storage and File Compression

1.3.1 Data Storage Measurement

> ⚠️ **IMPORTANT**
> Only the **IEC (binary) system** is in the Cambridge syllabus. Use powers of 2, not decimal (denary).

Unit	Calculation	Bytes
1 Kibibyte (KiB)	2^{10}	1,024 bytes
1 Mebibyte (MiB)	2^{20}	1,048,576 bytes
1 Gibibyte (GiB)	2^{30}	1,073,741,824 bytes
1 Tebibyte (TiB)	2^{40}	1,099,511,627,776 bytes
1 Pebibyte (PiB)	2^{50}	1,125,899,906,842,624 bytes

Why Binary System?

- Computer memory is based on powers of 2
- More accurate than decimal (denary) system
- Standard for RAM, ROM, and storage devices

> ⚠️ **KEY DIFFERENCE**
> **1 GiB** (binary) = 1,073,741,824 bytes ≠ **1 GB** (decimal) = 1,000,000,000 bytes

1.3.2 Data Compression

Why Compression Is Needed

- Large files take up storage space
- Slow download/upload speeds
- Expensive data transmission (bandwidth costs)
- Limited device storage

Lossless Compression

📘 **DEFINITION**

Reduces file size while retaining **ALL** original data. Original file can be perfectly reconstructed. No data loss.

🔑 **TECHNIQUE: RUN LENGTH ENCODING (RLE)**

How it works: Identifies sequences of repeated data. Stores as (count, value) instead of repeating the value.

TEXT EXAMPLE

```
Original: AAAAABBBCCCD
RLE:      5A 3B 2C 1D
```

IMAGE EXAMPLE (GREY=0, WHITE=1)

```
Original: 0000111100010000
RLE: 4×0, 4×1, 2×0, 1×1, 4×0
```

Advantages	Disadvantages
Perfect recovery of original data	Lower compression ratio than lossy
Suitable for text, documents, critical data	Not effective with complex/varied data
Works well with repetitive data	Decompression required to view file

Lossy Compression

📘 **DEFINITION**

Reduces file size by removing some data. Original file **CANNOT** be perfectly reconstructed. Data loss is permanent but often acceptable.

🔑 **WHEN USED & HOW IT WORKS**

When used: Images (JPEG), Audio (MP3), Video (MP4, H.264)

How it works: Removes data that human senses cannot perceive — e.g. colours eyes cannot distinguish or sound frequencies humans cannot hear.

Advantages	Disadvantages
Much higher compression ratio than lossless	CANNOT recover original data
Suitable for images, audio, video	Visible/audible quality loss if compressed too much
Smaller file sizes for large media files	Unsuitable for text and critical data

Choosing Compression Type

Content Type	Recommended	Reason
Text documents	**Lossless** (RLE)	Cannot lose text data
Software / executables	**Lossless**	Data integrity critical
Photographs	**Lossy** (JPEG)	Eyes cannot detect small colour differences
Web images	Lossy or Lossless	Logos → lossless, photos → lossy
Music	**Lossy** (MP3)	Ears cannot detect removed high frequencies
Videos	**Lossy**	High compression saves bandwidth

Quick Revision Sheet — Highest-Yield Points

🔑 NUMBER SYSTEMS

- Binary: only 1s and 0s (physical switches ON/OFF)
- 1 hex digit = 4 binary digits
- Hexadecimal used for: error codes, MAC addresses, IPv6, HTML colours
- Max 8-bit positive number: 255; min: 0

🔑 BINARY ARITHMETIC

- 1 + 1 = 0 (carry 1)
- Overflow: result requires more bits than available
- Left shift ×2; right shift ÷2

🔑 TWO'S COMPLEMENT

- Leftmost bit = −128 (not +128)
- Leftmost 1 = negative; leftmost 0 = positive
- Range 8-bit: −128 to +127
- Invert bits + add 1 = convert sign

🔑 SOUND & IMAGES

- Sampling rate (Hz): higher = better quality = larger file
- Sampling resolution (bits): higher = more amplitude levels = larger file
- CD standard: 44.1 kHz, 16-bit
- Colour depth: more bits = more colours = larger file
- Resolution: more pixels = more detail = larger file

🔑 DATA STORAGE

- Use IEC system (powers of 2): KiB, MiB, GiB, etc.
- NOT decimal system (KB, MB, GB)
- 1 MiB = 1,048,576 bytes (NOT 1,000,000)

🔑 COMPRESSION

- Lossless: keeps all data (RLE, text files, software)
- Lossy: removes data (JPEG, MP3, video)

Exam Tips and Common Mistakes

Binary Conversion Mistakes

⚠️ **COMMON ERROR**

Forgetting to include leading zeros for 8-bit numbers.

```
Wrong: 10110
```

```
Correct: 00010110
```

💡 **SOLUTION**

Always specify bit length (8-bit, 16-bit) and include leading zeros.

⚠️ **COMMON ERROR**

Adding binary as if it were decimal (1 + 1 = 2).

```
Correct: 1 + 1 = 0 with carry 1
```

Hexadecimal Mistakes

⚠️ **COMMON ERROR**

Forgetting that A–F represent 10–15.

💡 **SOLUTION**

Always write the conversion (A = 10, F = 15).

⚠️ **COMMON ERROR**

Incorrect grouping when converting binary to hex.

💡 **SOLUTION**

Always group from the RIGHT, pad left with zeros if needed.

File Size Calculation Errors

⚠️ **COMMON ERRORS**

- Forgetting to divide by 8 to convert bits to bytes
- Using decimal units (KB) instead of binary units (KiB)
- Forgetting to divide by 1,048,576 for MiB (not 1,000,000)

💡 **SOLUTION**

Work in bits first, then convert step-by-step to bytes, then to KiB/MiB.

Two's Complement Mistakes

⚠️ **COMMON ERROR**

Forgetting that range is −128 to +127 (not −255 to 0).

💡 **SOLUTION**

Always label the column headings (including −128, not +128).

Compression Mistakes

⚠️ **COMMON ERROR**

Saying RLE (lossless) is suitable for all file types. **Problem:** terrible compression for complex images and audio.

⚠️ **COMMON ERROR**

Using lossy compression for text/software. **Problem:** data loss makes files unusable.

💡 **SOLUTION**

Match compression type to content.

Final Reminders for Exam Day

- ✅ Always show working — marks awarded for method, not just answer
- ✅ Label binary numbers — specify 8-bit, 16-bit, etc.
- ✅ Use correct terminology — MiB not MB, denary not decimal
- ✅ Double-check conversions — especially hex (A–F values)
- ✅ State assumptions — e.g. "Assuming 8-bit unsigned binary"
- ✅ Comment on results — overflow, loss of precision, etc.
- ✅ Verify calculations — convert back to check answer
- ✅ Read questions carefully — lossless vs. lossy, MiB vs. KiB
- ✅ Practice binary addition — carries are easy to miss
- ✅ Understand WHY — not just the process, but the concept

These notes cover all examinable content for Cambridge O Level Computer Science (2210) Chapter 1: Data Representation. Use alongside Cambridge past papers for targeted exam practice.